



Cahier des Charges

Version 2.3

Plateforme de gestion locative — Chambres, maisons, magasins et locaux commerciaux

Compléments fonctionnels · Architecture technique · Modèle économique · Modèle de données

DATE
Septembre 2026

ZONE CIBLE
Guinée → Afrique de l'Ouest

STATUT
Schéma de données testé sur PostgreSQL 16

Sommaire



Analyse du document v1.0

Points forts et lacunes du cahier des charges original, avant tout ajout.



Partie A — Fonctionnalités

15 fonctionnalités manquantes ajoutées : garants, état des lieux, portail propriétaire, commission d'agence, prospection, super-admin...



Partie B — Architecture technique

Stack technique, choix de multi-tenance, sécurité, paiements mobiles, hors-ligne : l'architecture complète, justifiée.



Partie C — Priorisation

Impact, effort et priorité (MVP / V2 / V3 / Enterprise) de chaque ajout, en un coup d'œil.



Partie D — Modèle économique

Grille tarifaire, risque réglementaire de la commission sur paiements, indicateurs SaaS à suivre.



Partie E — Modèle de données

36 tables PostgreSQL documentées et testées : schéma exécutable, isolation multi-tenant vérifiée.



Conclusion

Ce qu'il faut trancher avant d'écrire la moindre ligne de code.

🔍 Analyse du document v1.0

Points forts

- Couverture fonctionnelle large et cohérente : Authentification → Propriétés → Locaux → Locataires → Contrats → Facturation → Paiements → Impayés.
- Approche SaaS multi-tenant pensée dès la conception, avec isolation des données par organisation.
- Bonne prise en compte du contexte local : GNF, Orange Money/MTN MoMo, connectivité instable, français comme langue principale.
- Découpage en phases (MVP → V2 → V3 → Enterprise) réaliste, priorisé sur la valeur métier.
- Modèle de rôles/permissions déjà granulaire (Owner / Admin / Manager / Agent / Tenant).

Lacunes fonctionnelles identifiées

Manque	Pourquoi c'est important
Garants / cautions solidaires	Quasi systématiques dans les baux en Afrique francophone
État des lieux (entrée/sortie)	Pièce centrale de tout litige sur la caution
Portail propriétaire	Le document cible des agences gérant <i>pour le compte de</i> tiers ; la question « combien doit-on lui verser ? » (section 2) n'a pas de module dédié
Commission d'agence	Aucune automatisation de la répartition agence/propriétaire
Pénalités de retard	Le retard est classifié, mais jamais chiffré financièrement
Prospection / visites	Rien entre « local disponible » et « locataire signé »
Signature électronique & import Excel	Le problème n°1 cité en introduction est justement l'usage d'Excel/papier — aucun outil de migration ni de signature en ligne
Gestion des prestataires	Simple champ texte « responsable », pas d'annuaire ni d'historique
Mode hors-ligne	Limité aux seuls paiements (section 38)

Lacunes techniques identifiées

- Stratégie de multi-tenance **non tranchée** (schéma partagé ? par client ?) — décision structurante à prendre avant tout développement.
- Aucune mention de cache, file de tâches asynchrones, stockage objet, observabilité, CI/CD ou hébergement.
- Aucun détail sur l'intégration technique réelle des paiements mobiles (webhooks, idempotence, réconciliation).
- Aucune stratégie de synchronisation hors-ligne (résolution de conflits).
- Aucun choix explicite sur la souveraineté des données.

Les deux parties suivantes traitent ces points un par un.



Partie A — Fonctionnalités

A1. Garants et cautions solidaires

Objectif : associer un ou plusieurs garants à un contrat. **Champs** : nom, téléphone, adresse, profession, pièce d'identité, lien avec le locataire, montant garanti, document signé. **Règle** : alerte si un contrat marqué « garant exigé » n'en a aucun.

A2. État des lieux (entrée / sortie)

Objectif : documenter l'état du local pour arbitrer objectivement les litiges sur la caution. **Champs** : type (entrée/sortie), date, état par pièce, photos horodatées, signature électronique locataire + bailleur. **Lien avec la caution** : à la sortie, le système compare automatiquement l'état d'entrée et de sortie et propose une répartition de la caution (retenue pour dégâts vs remboursement), reliée aux dépenses de remise en état enregistrées dans le module Dépenses.

A3. Portail propriétaire + relevé de compte

Objectif : donner à chaque propriétaire tiers (celui qui confie ses biens à une agence) une vue autonome de son patrimoine, sans accès aux fonctions opérationnelles. **Contenu** : loyers perçus, commission prélevée, dépenses imputées, montant net à verser, historique des versements, relevés PDF mensuels téléchargeables. **Rôle** : nouveau rôle « Owner-viewer » (lecture seule, restreint à ses propres biens), distinct du rôle Owner opérationnel déjà défini section 22.

A4. Commission d'agence et répartition des revenus

Objectif : automatiser le calcul de la commission de gestion prélevée par l'agence sur chaque loyer encaissé pour le compte d'un tiers. **Paramètres** : taux (fixe ou variable par bien/propriétaire), base de calcul (loyer brut ou net de dépenses), fréquence de reversement. **Sortie** : bordereau de versement propriétaire + écriture de commission agence, consultable dans le portail propriétaire (A3).

A5. Pénalités de retard automatiques

Objectif : appliquer une pénalité financière selon des règles configurables (ex. 2 % après J+7), au lieu d'une simple classification qualitative. **Paramètres** : mode de calcul (fixe, pourcentage, dégressif), plafond éventuel, configurable par organisation ou par contrat. **Lien** : le montant affiché dans le module Impayés (section 16) doit intégrer la pénalité calculée, pas seulement le loyer initial.

A6. Prospection et visites

Objectif : gérer le cycle « bien vacant → visite → candidature → sélection → contrat », absent du document alors que « locaux disponibles » est un indicateur clé du tableau de bord (section 8). **Champs** : fiche prospect (contact, budget, biens visités), planification de visite (intégrée au calendrier existant, section 18), conversion en locataire en un clic.

A7. Signature électronique

Objectif : signer contrats et états des lieux sans imprimer. **Options** : intégration tierce (Yousign, DocuSign) ou solution minimale interne (capture de signature manuscrite + horodatage + hash du document pour une valeur probante de base).

A8. Import / migration de données

Objectif : réduire la friction d'onboarding — le problème n°1 cité dans le contexte (section 1.1) est l'usage d'Excel et de cahiers. **Fonctionnalité** : assistant d'import CSV/Excel avec mapping de colonnes, détection de doublons, validation avant import définitif, pour propriétés, locaux, locataires et contrats existants.

A9. Assistant conversationnel WhatsApp

Objectif : permettre aux agents et propriétaires peu à l'aise avec une interface web d'effectuer des actions clés via un canal déjà omniprésent en Guinée. **Portée MVP du bot** : « Quel est le solde de Mamadou ? », « Paiement de 500 000 GNF reçu de Fatoumata pour le Local M-003 ». **Complément naturel** du canal WhatsApp déjà prévu section 17 pour les notifications sortantes — ici en entrée également.

A10. Gestion des prestataires de maintenance

Objectif : remplacer le champ libre « responsable » (section 19) par un véritable annuaire de prestataires (plombier, électricien, peintre...) avec historique d'interventions, coût moyen, délai moyen et notation interne.

A11. Indexation et révision des loyers

Objectif : gérer les augmentations périodiques (annuelles, à la reconduction) selon une règle (taux fixe, indice, décision manuelle), avec conservation de l'historique des montants successifs par contrat.

A12. Conformité légale locale

Objectif : signaler explicitement que les règles métier sensibles (durée de préavis, plafond légal de caution en mois de loyer, motifs de résiliation) doivent être validées avec un juriste guinéen avant d'être fixées par défaut, et rendues **configurables** plutôt que codées en dur.

A13. Mode hors-ligne étendu

Objectif : étendre le hors-ligne — aujourd'hui limité aux paiements (section 38) — à la consultation des fiches locataires/contrats et à la création de tickets de maintenance, cas d'usage fréquent pour un agent sur le terrain sans réseau.

A14. Vue consolidée propriétaire multi-agence (*Enterprise*)

Objectif : pour un propriétaire confiant des biens à plusieurs agences utilisatrices de la plateforme, une vue consolidée cross-organisation. À ne considérer qu'en V3/Enterprise, une fois le socle multi-tenant éprouvé.

A15. Administration de la plateforme (Super-Admin)

Constat : la section 27 du document v1.0 esquisse un « espace super-admin » en une liste de puces, sans jamais le développer — ni rôle défini, ni interaction précisée avec l'isolation multi-tenant (B3), ni table dans le modèle de données. C'est un manque significatif : c'est l'éditeur de la plateforme lui-même qui en a besoin, dès le premier client payant.

Pourquoi ce n'est pas « juste un rôle de plus » : Owner/Admin/Manager/Agent (section 22) sont tous scopés à une organisation via RLS. Le Super-Admin doit voir et agir sur **toutes** les organisations — l'ajouter comme rôle dans la table `users` existante casserait la sémantique même de l'isolation qu'on vient de construire (B3, B6). Il vit dans son propre espace, hors du périmètre multi-tenant.

Fonctionnalités (précise et structure la section 27) :

- Gestion des organisations clientes : création manuelle, suspension, réactivation, résiliation, changement de plan forcé (geste commercial).
- Vue consolidée des abonnements et de la facturation SaaS : MRR/ARR/churn agrégés — les indicateurs de D4, mais à l'échelle de toute la plateforme, pas d'une seule organisation.
- Gestion du catalogue de plans (créer/modifier les paliers Free/Standard/Pro/Business et leurs limites — D1).
- File de tickets support, avec assignation et priorité.
- **Impersonation encadrée** : se connecter « en tant que » une organisation cliente pour déboguer un problème signalé, chaque session tracée (qui, quand, pourquoi, quelles actions).
- Journal d'audit dédié aux actions super-admin, distinct du journal d'activité par organisation (section 24).
- Validation manuelle des demandes de passage en mode agrégateur de paiement (D3) — une décision de conformité réglementaire, qui ne devrait jamais être un self-service automatique.

Rôles internes (moindre privilège) : *Support* (lecture + impersonation, pas de facturation), *Billing-Admin* (abonnements/plans), *Super-Admin* (accès complet).



Partie B — Architecture technique

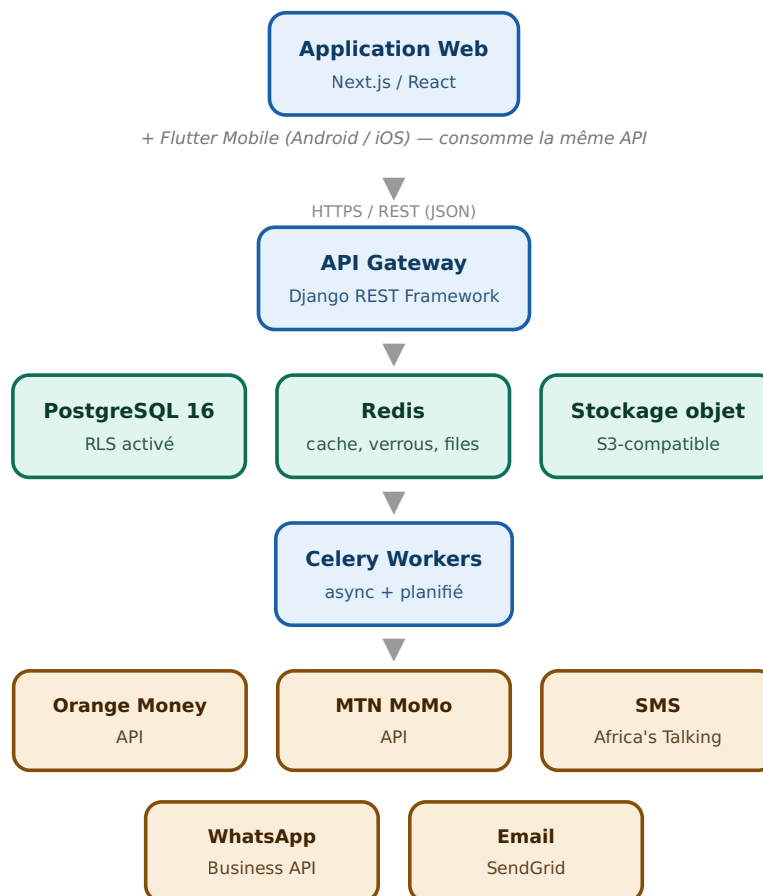
B1. Vue d'ensemble et choix architectural

Recommandation : démarrer en monolithe modulaire, et non en microservices.

Un seul déploiement Django, organisé en apps fortement découplées par domaine (`billing` , `tenants` , `maintenance` , `notifications` , `payments` ...), chacune avec ses propres modèles, vues et tests. Ce découpage logique permet d'extraire un domaine en microservice indépendant plus tard, sans réécriture complète.

Pourquoi pas de microservices dès le départ : la complexité opérationnelle (orchestration, observabilité distribuée, cohérence transactionnelle inter-services) n'est pas justifiée pour une équipe de démarrage, et ralentirait le MVP sans bénéfice réel à ce stade.

Trajectoire d'évolution : le premier candidat à l'extraction est le service de **notifications** (volume élevé, nature asynchrone, peu couplé au reste), à isoler dès que la charge le justifie, via communication par file de messages plutôt que par appel synchrone.



B2. Stack technique recommandée

Couche	Technologie proposée	Justification
Frontend Web	Next.js (React) + TypeScript, Tailwind CSS	Rendu rapide sur connexions lentes, écosystème mature
Mobile	Flutter	Un seul code base iOS/Android, bon support hors-ligne (<code>drift</code> /SQLite)

Couche	Technologie proposée	Justification
Backend API	Django + Django REST Framework, Python 3.12	Productivité, admin intégrée, écosystème d'auth/permissions mature
Base de données	PostgreSQL 16	Row Level Security native (clé pour le multi-tenant), JSONB, extensions
Cache	Redis	Sessions, cache de requêtes, verrous distribués
File de tâches	Celery + Redis (broker)	Facturation périodique, notifications, PDF, pénalités
Recherche	Full-text search PostgreSQL (MVP) → OpenSearch si besoin plus tard	Éviter la complexité d'Elasticsearch avant que le volume ne le justifie
Stockage fichiers	S3-compatible (MinIO auto-hébergé ou OVH/AWS Object Storage)	Jamais en base — conforme à la section 30 du document initial
Authentification	JWT (access + refresh rotatif) + TOTP pour le 2FA	Stateless, adapté web + mobile
Paielements mobiles	API Orange Money, API MTN MoMo (Collections/Disbursements)	Marchés cibles
SMS	Africa's Talking ou Twilio	Bonne couverture Afrique de l'Ouest
WhatsApp	WhatsApp Business Cloud API (Meta)	Canal déjà utilisé localement
Email	SendGrid ou Mailgun	Délivrabilité
Observabilité	Sentry, Prometheus + Grafana, Loki	Standard éprouvé et léger
CI/CD	GitHub Actions + Docker	Automatisation tests/déploiement
Infra as Code	Terraform	Reproductibilité des environnements
Orchestration	Docker Compose (MVP) → Kubernetes managé si croissance	Éviter la complexité K8s prématurément

B3. Multi-tenance : la décision structurante

Trois options, à trancher **avant** tout développement :

Stratégie	Isolation	Coût opérationnel	Recommandation
Base de données par client	Maximale	Élevé (migrations xN, sauvegardes xN)	Réservé à de très gros comptes Enterprise
Schéma PostgreSQL par client	Bonne	Moyen, complexe à automatiser à grande échelle	Option de repli pour un client exigeant
Schéma partagé + <code>organization_id</code> + Row Level Security	Bonne, appliquée au niveau base	Faible	Recommandé pour le MVP et la cible « plusieurs milliers de propriétés »



Avec la RLS PostgreSQL, même une erreur applicative (requête Django mal filtrée) ne peut pas exposer les données d'une autre organisation — la base l'interdit elle-même. C'est une défense en profondeur en complément du filtrage côté code.

B4. Modèle de données enrichi

En complément du schéma de la section 28 du document initial, ajouter :

guarantors	→ rattaché à contracts
inspections	→ rattaché à contracts (entrée / sortie)
inspection_items	→ rattaché à inspections
agency_commission_rules	→ rattaché à organizations / properties
owner_payouts	→ rattaché à properties / organizations
late_fee_rules	→ rattaché à organizations / contracts
leads	→ prospects avant conversion en tenants
vendors	→ prestataires, rattaché à maintenance_requests
esignatures	→ rattaché à contracts / inspections
rent_revisions	→ historique des montants, rattaché à contracts

B5. Conception de l'API

- Documentation générée automatiquement (OpenAPI/Swagger via `drf-spectacular`).
- Versionnement explicite dès le départ : `/api/v1/...` — pour ne jamais casser l'app mobile en production lors d'une évolution.
- Pagination systématique, y compris pour les listes qui semblent petites au MVP (paiements, journal d'activité) : la volumétrie grossit vite.
- Limitation de débit (rate limiting) par utilisateur/IP pour prévenir les abus.
- Format d'erreur normalisé (code, message, champ concerné) partagé entre web et mobile.

B6. Authentification et sécurité renforcée

- JWT access court (≈15 min) + refresh token rotatif, révocation possible via liste noire Redis.
- 2FA TOTP (compatible Google Authenticator), déjà prévu section 7, précisé techniquement ici.
- Double barrière : RBAC applicatif (rôles) + RLS PostgreSQL (organisation).
- Secrets injectés par variable d'environnement au déploiement, jamais commités ; solution de type Vault si le budget le permet.
- Chiffrement applicatif des champs les plus sensibles (numéro de pièce d'identité) en complément du chiffrement disque.
- Analyse automatique des dépendances (Dependabot) et tests de sécurité statiques (`bandit`) intégrés en CI.
- Test d'intrusion externe recommandé avant le passage en V3/Enterprise.

B7. Traitement asynchrone et planification

- Celery + Redis pour : génération périodique des factures, envoi des notifications programmées (J-7/J-3/J/J+1/J+7, déjà définies section 41), génération de PDF, calcul des pénalités de retard.
- Celery Beat pour les tâches planifiées récurrentes.
- Réessai automatique avec backoff exponentiel pour les tâches critiques (ex. envoi SMS).

B8. Cache et performance

- Redis pour : cache des indicateurs du tableau de bord (invalidation à l'écriture ou rafraîchissement périodique), verrous distribués — utile pour éviter qu'un même paiement soit enregistré deux fois en cas de double clic ou de synchronisation hors-ligne concurrente.
- Index PostgreSQL ciblés (`organization_id`, `status`, `due_date`) et usage systématique de `select_related` / `prefetch_related` côté Django dès le MVP, pas seulement « en cas de besoin » comme suggéré section 37.

B9. Stockage de fichiers

- Bucket S3-compatible, un préfixe par organisation.
- URLs signées à durée limitée pour les documents sensibles (pièces d'identité).
- Compression et redimensionnement automatiques des photos (maintenance, biens) pour économiser la bande passante mobile — pertinent vu la connectivité ciblée.

B10. Intégrations de paiement mobile money

- Orange Money et MTN MoMo : intégration via leurs API Collections — le locataire confirme le paiement (push USSD), un webhook notifie la plateforme.
- **Idempotence obligatoire** sur la réception des webhooks (clé unique de transaction) pour éviter un double enregistrement en cas de nouvelle tentative réseau.
- Réconciliation quotidienne automatique entre les transactions déclarées par l'opérateur et celles enregistrées côté plateforme, avec alerte en cas d'écart.

B11. Notifications multi-canal

- Firebase Cloud Messaging pour le push mobile.
- Africa's Talking ou Twilio pour le SMS.
- WhatsApp Business Cloud API.
- SendGrid/Mailgun pour l'email.
- Le moteur de règles déjà prévu section 41 doit choisir le canal selon la préférence de l'utilisateur et sa disponibilité (repli SMS si pas de smartphone).

B12. Mode hors-ligne mobile

- Base locale embarquée (SQLite via `drift`) répliquant un sous-ensemble pertinent pour l'agent : locataires, contrats actifs, soldes.
- File d'attente locale des actions (paiement, ticket créé), rejouée à la reconnexion.
- Résolution de conflits : « dernière écriture serveur gagne » pour les champs simples ; les paiements, eux, sont **toujours additifs** — jamais de conflit destructif, un paiement enregistré hors-ligne vient s'ajouter, jamais écraser une donnée serveur.

B13. Observabilité

- Sentry pour la capture d'erreurs front et back en temps réel.
- Prometheus + Grafana pour les métriques techniques (latence API, taux d'erreur, charge base de données).
- Logs centralisés (Loki) avec politique de rétention définie.
- Point de santé (`/health`) exposé par chaque service, utilisé par l'orchestrateur.

B14. CI/CD et Infrastructure as Code

- Pipeline : lint → tests unitaires → tests API → build image Docker → déploiement automatique en staging → déploiement manuel validé en production.
- Terraform pour décrire l'infrastructure (base managée, bucket, réseau) de façon reproductible entre environnements.

B15. Hébergement et souveraineté des données

- Deux options à trancher avec le porteur de projet : hébergement régional (fournisseur le plus proche d'Afrique de l'Ouest disponible) vs cloud international.
- **Point d'attention juridique** : la réglementation guinéenne/CEDEAO sur la protection des données personnelles peut imposer des contraintes de localisation — à valider avant le choix définitif de l'hébergeur, en particulier pour les pièces d'identité et données financières évoquées section 35.
- Recommandation transitoire pour le MVP : cloud international fiable (meilleur SLA, coût de démarrage plus faible), avec une architecture (voir B3, B9) permettant une migration vers un hébergement local si la réglementation l'exige.

B16. Scalabilité et montée en charge

- Répliques de lecture PostgreSQL lorsque la charge des rapports/dashboard dépasse la capacité du serveur primaire.
- CDN (type Cloudflare) pour les assets statiques et les photos — important vu la connectivité variable ciblée par le projet.
- Découplage des tâches lourdes (rapports PDF volumineux) en tâches asynchrones plutôt qu'en requêtes HTTP synchrones bloquantes.

B17. Roadmap technique par phase

Phase	Ajouts techniques
MVP	Monolithe modulaire Django + Next.js, PostgreSQL avec RLS dès le départ , Redis, stockage S3-compatible, CI/CD basique, Sentry
V2	Celery pour l'asynchrone, intégration paiement mobile money (webhooks + réconciliation), observabilité complète
V3	App Flutter avec mode hors-ligne, WhatsApp Business API, CDN, répliques de lecture
Enterprise	Option schéma-par-client pour clients exigeants, Kubernetes si la charge le justifie, audit de sécurité externe

B18. Architecture de l'administration plateforme (Super-Admin)

Isolation vis-à-vis du RLS : le super-admin n'a pas d' `organization_id` — le forcer casserait la sémantique du multi-tenant décidée en B3. Deux options :

1. **Table séparée** `platform_admins`, hors du schéma multi-tenant, avec son propre backend/API (`/api/admin/...`) utilisant un rôle PostgreSQL dédié avec `BYPASSRLS` — jamais partagé avec le rôle applicatif tenant-facing.
2. (*Déconseillé*) Un `organization_id` sentinelle « plateforme » — risque de fuite accidentelle si une politique RLS est mal écrite ailleurs.



Recommandation MVP : ne pas construire d'interface admin sur-mesure dès le départ. Le monolithe modulaire Django déjà retenu (B1) inclut Django Admin gratuitement — connecté avec un rôle PostgreSQL `BYPASSRLS`, il couvre 80 % du besoin initial (lister les organisations, suspendre un compte, ajuster un abonnement) sans une ligne de frontend à écrire. Réserver le développement d'une vraie interface super-admin (impersonation, dashboard MRR) à la V2, une fois le besoin confirmé par l'usage réel.

Sécurité renforcée, non négociable sur ce périmètre :

- 2FA **obligatoire** (pas optionnel comme pour les utilisateurs organisationnels, section 7).
- Accès restreint par IP allowlist ou VPN pour le backend admin.
- Chaque action journalisée dans `platform_audit_logs`, y compris les lectures sensibles (impersonation) — jamais uniquement dans le `activity_logs` par organisation (section 24), qui n'est pas conçu pour ça.
- Séparation des environnements : le service admin peut tourner comme une app Django distincte (même base de code, déploiement séparé) pour réduire la surface d'attaque exposée publiquement.

Impersonation technique : générer un JWT de courte durée (5-15 min) scopé à l'organisation cible, avec un claim `impersonated_by=<admin_id>` que le middleware RLS utilise pour fixer `app.current_org_id` normalement, mais que le frontend affiche en bandeau visible (« Vous consultez le compte de X en tant que support ») pour éviter toute confusion pour l'agent support lui-même.

☰ Partie C — Priorisation

Ajout	Impact métier	Effort	Priorité
Portail propriétaire + commission agence (A3, A4)	Très élevé	Moyen	MVP
Pénalités de retard (A5)	Élevé	Faible	MVP
Décision multi-tenance RLS (B3)	Structurant	Faible si pris tôt	MVP — non négociable
Espace Super-Admin minimal via Django Admin (A15, B18)	Structurant — indispensable dès le 1er client	Faible	MVP
Garants (A1)	Élevé	Faible	MVP
État des lieux (A2)	Élevé	Moyen	V2
Import Excel (A8)	Élevé pour l'adoption	Moyen	V2
Prospection/visites (A6)	Moyen	Moyen	V2
Intégration mobile money (B10)	Très élevé	Élevé	V2
Interface Super-Admin enrichie : impersonation, dashboard MRR (A15, B18)	Élevé une fois plusieurs agences actives	Moyen	V2
Bot WhatsApp (A9)	Moyen, fort différenciateur local	Élevé	V3
Signature électronique (A7)	Moyen	Moyen	V3
Mode hors-ligne étendu (A13, B12)	Élevé pour les agents terrain	Élevé	V3
Vue multi-agence (A14)	Faible à court terme	Élevé	Enterprise



Partie D — Modèle économique

D0. Synthèse de l'analyse des pistes de monétisation proposées

Points forts de la réflexion initiale

- Diversité des leviers identifiés : abonnement, feature-gating, notifications, commission sur paiements, services professionnels, offre agences, marketplace, statistiques premium, assistant IA.
- Bonne intuition produit : ne pas monétiser dix choses à la fois, se concentrer d'abord sur la boucle de valeur centrale (Propriété → Local → Locataire → Contrat → Échéance → Paiement → Reçu → Impayé).
- Séquençage réaliste en quatre phases (traction → valeur → agences → paiements).
- Segmentation particuliers/agences pertinente : leur valeur vie client (LTV) est de nature complètement différente.

Points à muscler avant de figer un pricing

Manque	Pourquoi c'est important
Risque réglementaire de la commission sur paiements non traité	Un logiciel qui prélève un pourcentage sur des fonds qui transitent peut être qualifié d'intermédiaire financier
Prix (50 000 / 100 000 / 250 000 GNF) non validés	Ce sont des hypothèses de travail, pas des données de marché
Palier gratuit potentiellement trop généreux	Beaucoup de petits propriétaires guinéens gèrent précisément 1 maison de quelques chambres — le palier gratuit couvrirait déjà tout leur besoin, sans limite de temps
Incohérence de nommage des paliers avec la section 26 du document v1.0	v1.0 utilise Free/Pro/Business (3 paliers), la réflexion en propose 4 (Gratuit/Standard/Pro/Business)
Aucun indicateur de pilotage défini	Impossible de savoir si le modèle fonctionne sans MRR, churn, ARPU, CAC
Moyen de paiement de l'abonnement lui-même non traité	Le client cible paie plutôt en cash ou en Mobile Money — pas nécessairement par carte
Érosion monétaire du GNF non anticipée	Une grille tarifaire figée en GNF perd de la valeur réelle avec l'inflation

D1. Grille tarifaire harmonisée (précise la section 26 du document v1.0)

Palier	Cible	Limites indicatives	Fonctionnalités incluses
Free	Petit propriétaire, découverte	1 propriété, 3 locaux max (recalibré, voir D2)	Suivi de paiement basique, reçu simple sans PDF
Standard	Propriétaire avec quelques biens	Locaux étendus	Reçus PDF, rapports de base, historique, pénalités de retard (A5)
Pro	Propriétaire multi-biens / petit gestionnaire	Locaux illimités	Employés + permissions, notifications automatiques (quota inclus), état des lieux (A2), signature électronique (A7), portail propriétaire (A3)

Palier	Cible	Limites indicatives	Fonctionnalités incluses
Business	Agences immobilières	Multi-gestionnaires, multi-propriétaires	Commission d'agence automatisée (A4), API, comptabilité, statistiques avancées, bot WhatsApp (A9)

Un **prix validé** demande une méthode, pas seulement une intuition : entretiens de disposition à payer avec les 20 premiers prospects de la Phase 1, ou test de sensibilité au prix type Van Westendorp, avant de figer la grille définitive.

Pour lisser la trésorerie et réduire le churn, prévoir une **remise à l'engagement annuel** (ex. 2 mois offerts sur 12 payés d'avance) — un mode de paiement particulièrement adapté à un marché où l'abonnement mensuel automatique n'est pas encore un réflexe.

D2. Recalibrer le palier gratuit



Le risque du palier « 1 propriété / 5 locaux gratuits à vie » : c'est exactement le profil du propriétaire individuel le plus courant en Guinée, qui n'aura donc **jamais de raison de payer**. Deux options, à arbitrer selon les retours terrain de la Phase 1 :

1. Réduire le plafond gratuit (ex. 3 locaux au lieu de 5) pour qu'il fonctionne comme un vrai palier d'entrée plutôt qu'une solution définitive.
2. Ou transformer le « gratuit » en **essai limité dans le temps** (ex. 60 jours, toutes fonctionnalités), suivi d'un palier payant même minime — plus simple à faire comprendre, et plus efficace pour convertir.

D3. Commission sur les paiements : le potentiel réel et le risque à traiter en priorité



Attention — distinction importante : ne pas confondre la *commission d'agence* (fonctionnalité A4 — l'agence prélève une part du loyer pour le compte du propriétaire qu'elle gère) avec la *commission de la plateforme elle-même* sur le volume de paiements qui y transitent (revenu du logiciel, traité ici).



Le calcul de la réflexion initiale est juste : à 0,5 % sur un volume mensuel de quelques centaines de millions à quelques milliards de GNF, le revenu potentiel dépasse largement celui des abonnements seuls. **Mais prélever un pourcentage sur des fonds qui transitent qualifie généralement l'éditeur d'intermédiaire financier** (agrégateur de paiement), potentiellement soumis à autorisation de la Banque Centrale de la République de Guinée (BCRG) ou à un partenariat contractuel avec un émetteur de monnaie électronique agréé — Orange Money et MTN MoMo le sont déjà, une jeune plateforme SaaS ne l'est pas par défaut.

Deux postures possibles :



Mode facilitateur — recommandé pour démarrer



La plateforme déclenche et trace la transaction, facture des frais de service **fixes** — jamais un pourcentage prélevé sur le flux.



Mode agrégateur — risque réglementaire, phase avancée



Nécessite une licence ou un partenariat formalisé avec un opérateur agréé.



Recommandation : démarrer en mode facilitateur (frais de service fixes par transaction ou par facture, jamais un pourcentage prélevé à la source), et ne considérer le mode agrégateur qu'en V3/Enterprise, avec un conseil juridique dédié et un partenariat formalisé avec un opérateur agréé. Toute communication commerciale évoquant une « commission automatique » doit attendre que ce cadre soit sécurisé.

D4. Indicateurs de pilotage (*complète la section 45 du document v1.0*)

À suivre dès les premiers clients payants :

- **MRR** (revenu récurrent mensuel) et **ARR** (annualisé)
- **Taux de churn mensuel** (résiliations / total abonnés)
- **ARPU** (revenu moyen par client) — à suivre séparément pour particuliers et agences, leurs LTV sont incomparables
- **CAC** (coût d'acquisition client) et ratio **LTV/CAC**
- Taux de conversion Free → payant, et mensuel → engagement annuel
- Take rate moyen sur les paiements, une fois le mode facilitateur (D3) activé

D5. Paiement de l'abonnement lui-même

Le moyen le plus naturel pour la cible n'est pas nécessairement la carte bancaire :

- Prélèvement mensuel via l'API Orange Money / MTN MoMo (avec consentement explicite du client), pour réduire le churn « oubli de paiement »
- En Phase 1, accepter un règlement en espèces via un agent de terrain le temps que la confiance dans le paiement en ligne s'installe

D6. Indexation tarifaire face à l'inflation du GNF

Une grille figée en GNF perd de la valeur réelle avec le temps. Prévoir dès les conditions générales d'abonnement une clause de révision annuelle des prix, plutôt que de découvrir la nécessité d'augmenter les tarifs sans cadre contractuel préétabli.

D7. Stratégie de mise sur le marché — phases et critères de passage

La progression en quatre phases proposée est solide ; il lui manquait des critères chiffrés pour savoir quand passer à la phase suivante :

Phase	Objectif	Prix indicatif	Critère de passage à la phase suivante
1 — Traction	20 à 50 propriétaires payants ; valider la disposition à payer	Prix d'appel bas (~50 000 GNF/mois)	Churn mensuel < 10 % et premières recommandations spontanées
2 — Valeur	Ajout facturation auto, rappels, reçus, employés, rapports	100 000 – 250 000 GNF/mois	ARPU en hausse sans hausse du churn
3 — Agences	Offre Business dédiée aux gestionnaires multi-biens	500 000 – 1 500 000 GNF/mois	3 à 5 agences pilotes validant le prix Business
4 — Paiements	Intégration mobile money en mode facilitateur (D3)	Frais fixes par transaction	Cadre réglementaire clarifié avant d'envisager le mode agrégateur

D8. Catalogue de services professionnels (*complète le point « services supplémentaires » de la réflexion*)

Tarifier séparément les prestations ponctuelles qui financent l'onboarding sans attendre la conversion complète en abonnement : configuration initiale, migration des données existantes (lié techniquement au module A8), formation des équipes, accompagnement au changement pour les agences. Ce contact humain payant, en plus de générer un revenu immédiat, réduit le churn précoce en sécurisant les premières semaines d'usage.

D9. Ce qu'il ne faut pas faire tout de suite



L'avertissement de la réflexion initiale reste le plus important de toute cette partie : ne pas construire dix leviers de revenu en parallèle. Ordre recommandé : **(1)** abonnement simple avec palier gratuit recalibré (D2), **(2)** reçus PDF et rapports payants, **(3)** notifications avec quota, **(4)** offre agence dédiée, **(5)** paiements en mode facilitateur (D3). La marketplace immobilière et l'assistant IA restent des paris de phase avancée — ne pas les mélanger avec la validation du problème central tant qu'elle n'est pas acquise.



Partie E — Modèle de données

Cette partie documente le schéma PostgreSQL qui porte l'ensemble des Parties A, B et D. Elle explique les choix de conception et le rôle de chaque table ; le détail exhaustif de chaque colonne, contrainte et index vit dans le fichier exécutable `schema.sql` qui accompagne ce document — **testé sur une instance PostgreSQL 16 réelle**, résultats en E7.

E0. Principes de conception

- **Clés primaires en UUID**, jamais en entier auto-incrémenté : un identifiant séquentiel laisse deviner le volume d'activité d'un client à un concurrent (`/api/v1/contracts/4821`), et complique la synchronisation hors-ligne (B12) — deux agents créant chacun un contrat sans réseau ne peuvent pas se disputer le même identifiant suivant.
- **organization_id dupliqué sur chaque table métier**, même lorsqu'il est déductible par jointure (`units` → `properties` → `organization_id`). Une politique RLS ne filtre efficacement qu'une colonne présente directement sur la table lue ; la dupliquer transforme le filtre en une simple égalité indexée, sans jointure ni sous-requête à chaque lecture.
- **Cohérence garantie par triggers**, pas par discipline applicative : un trigger `BEFORE INSERT` recopie automatiquement `organization_id` depuis la table parente (`property_id`, `unit_id`, `contract_id`, `invoice_id` ...). Il devient impossible d'insérer une ligne avec un `organization_id` incohérent, même par erreur de code côté Django.
- **Deux notions de « facture » explicitement séparées** : `invoices` (loyer dû par le locataire, section 13 du document v1.0) et `platform_invoices` (abonnement SaaS dû par l'organisation cliente, Partie D). Les confondre dans une seule table aurait été une source de bugs garantie.
- **Suppression logique séparée de l'isolation** : `deleted_at` (sur `properties`, `units`, `tenants`) reste un filtre applicatif (Django manager par défaut), volontairement distinct de la politique RLS, pour qu'un compte de support avec le bon `organization_id` puisse encore consulter une fiche archivée.
- **Isolation « fail closed »** : si le contexte `app.current_org_id` n'est jamais fixé sur une requête (oubli du middleware), la politique RLS ne renvoie aucune ligne plutôt que de lever une erreur ou, pire, de tout exposer. Comportement vérifié par un test réel, voir E7.
- **Journal d'activité en écriture seule** (`activity_logs`) : aucune mise à jour ni suppression applicative, conformément à la section 24 du document v1.0.

E1. Inventaire complet des tables

Domaine	Table	Rôle
Tenancy & utilisateurs	<code>organizations</code>	La frontière multi-tenant : une agence ou un propriétaire indépendant
	<code>users</code>	Comptes de connexion (owner, admin, manager, agent, owner_viewer)
	<code>organization_members</code>	Accès multi-organisation (agent multi-agences, propriétaire multi-agence — A14)
Patrimoine	<code>owners</code>	Le propriétaire comme entité métier, distinct de son éventuel compte <code>users</code>
	<code>properties</code>	Les propriétés (section 9 du document v1.0)
	<code>units</code>	Les locaux : chambres, magasins, bureaux (section 10)
Location	<code>tenants</code>	Fiches locataires (section 11)

Domaine	Table	Rôle
	leads	Prospects avant conversion en locataire (A6)
	property_visits	Visites planifiées d'un local par un prospect (A6)
	contracts	Le contrat de location (section 12)
	guarantors	Garants rattachés à un contrat (A1)
	rent_revisions	Historique des révisions de loyer (A11)
	inspections	États des lieux d'entrée/sortie (A2)
	inspection_items	Détail pièce par pièce d'un état des lieux
	esignatures	Signatures électroniques de contrats ou d'états des lieux (A7)
Argent — loyers	invoices	Factures de loyer (section 13)
	payments	Paielements encaissés (section 14)
	late_fee_rules	Règles de pénalité de retard (A5)
Argent — agence	agency_commission_rules	Taux de commission de l'agence par bien ou propriétaire (A4)
	owner_payouts	Relevés et versements aux propriétaires (A3, A4)
Exploitation	vendors	Annuaire des prestataires de maintenance (A10)
	maintenance_requests	Tickets de maintenance (section 19)
	expenses	Dépenses (section 20)
Gouvernance	documents	Métadonnées des fichiers stockés en objet (section 30)
	notification_rules	Règles d'envoi automatique (section 41)
	notifications	File des notifications envoyées
	activity_logs	Journal d'activité, append-only (section 24)
Modèle SaaS	subscription_plans	Les 4 paliers Free/Standard/Pro/Business (Partie D1)
	subscriptions	Abonnement actif d'une organisation
	platform_invoices	Facturation de l'abonnement SaaS lui-même
	usage_counters	Consommation de quota (notifications, SMS)

Domaine	Table	Rôle
	payment_transactions	Traçabilité Orange Money / MTN MoMo, mode facilitateur (D3)

E2. Domaine 1 — La boucle centrale

C'est la boucle identifiée dans la conclusion du document v1.0 : *Propriété* → *Local* → *Locataire* → *Contrat* → *Facture* → *Paiement*. Elle doit fonctionner parfaitement avant tout le reste.

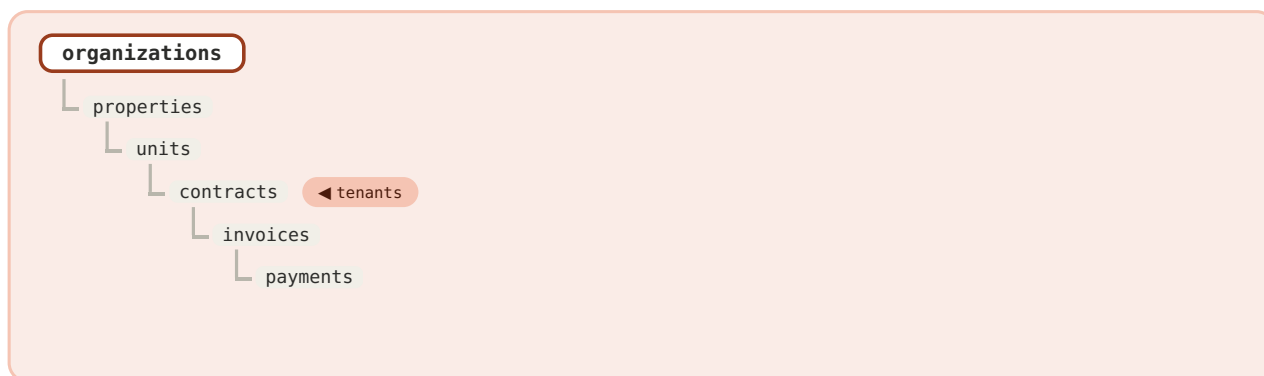


Table	Champs clés	Rôle
organizations	org_type , status	Racine de l'isolation multi-tenant
properties	owner_id , property_type , status	Une propriété appartient à un owner , jamais directement à un user
units	property_id , rent_amount , status	Statuts : disponible, réservé, occupé, maintenance, hors service (section 10)
tenants	id_doc_number , emergency_contact	Fiche locataire, indépendante du contrat pour conserver l'historique multi-contrats
contracts	unit_id , tenant_id , due_day , status	Le nœud central : relie un local à un locataire avec ses conditions
invoices	contract_id , due_date , late_fee_amount , status	Une facture par période ; late_fee_amount alimenté par late_fee_rules (A5)
payments	invoice_id , payment_method , recorded_by	Plusieurs paiements peuvent couvrir une même facture (paiement partiel)

E3. Domaine 2 — Extensions locatives (Partie A)

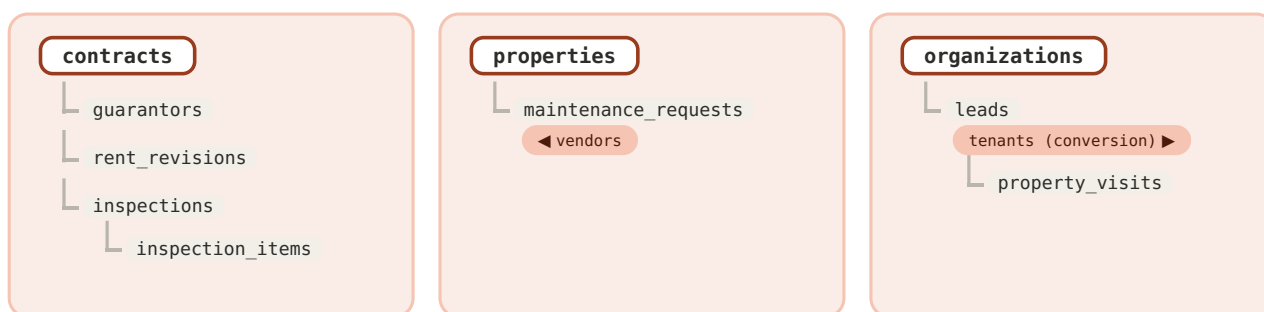


Table	Champs clés	Rôle
guarantors	contract_id , guaranteed_amount	Un contrat peut avoir plusieurs garants
rent_revisions	effective_date , previous_amount , new_amount	Historique complet, jamais d'écrasement du loyer d'origine
inspections	inspection_type (entry/exit), performed_by	Sert de base à la répartition de la caution en sortie
inspection_items	room_name , condition_rating , photo_urls	Une ligne par pièce inspectée
esignatures	signable_type , signable_id	Table polymorphe : signe un contrat ou un état des lieux
leads	status , converted_tenant_id	Pipeline new → visiting → negotiating → converted/lost
property_visits	lead_id , unit_id , scheduled_at	Alimente le calendrier (section 18)

E4. Domaine 3 — Argent agence & exploitation

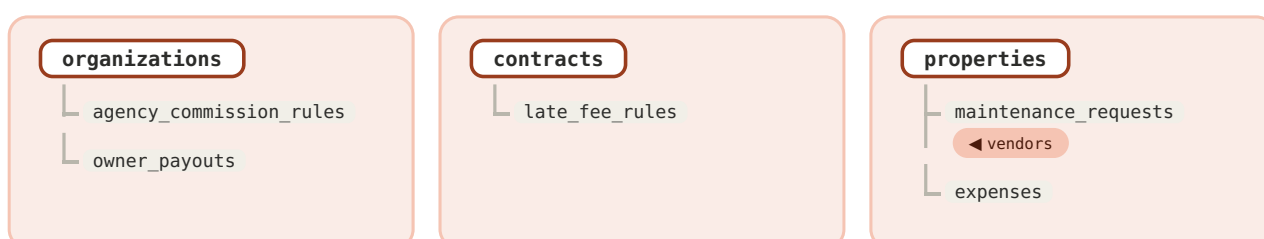


Table	Champs clés	Rôle
late_fee_rules	calculation_type , grace_period_days , cap_amount	Règle par contrat ou par défaut d'organisation (D3)
agency_commission_rules	rate_percentage , calculation_base	Taux par bien ou par propriétaire (A4)
owner_payouts	gross_rent_collected , commission_amount , net_amount	Le relevé consultable dans le portail propriétaire (A3)
vendors	specialty , average_cost , rating	Remplace le champ libre « responsable » de la section 19

Table	Champs clés	Rôle
<code>maintenance_requests</code>	<code>priority</code> , <code>vendor_id</code> , <code>status</code>	Cycle new → assigned → in_progress → resolved → closed
<code>expenses</code>	<code>category</code> , <code>property_id</code> , <code>receipt_url</code>	Alimente le calcul du bénéfice (section 21)

E5. Domaine 4 — Gouvernance & communication

Table	Champs clés	Rôle
<code>users</code>	<code>role</code> , <code>two_factor_enabled</code>	Rôles : owner, admin, manager, agent, owner_viewer (A3)
<code>organization_members</code>	<code>user_id</code> , <code>organization_id</code> , <code>role</code>	Table de jonction pour l'accès multi-organisation
<code>owners</code>	<code>user_id</code> (nullable)	<code>user_id</code> reste NULL tant que le portail propriétaire n'est pas activé
<code>documents</code>	<code>related_type</code> , <code>related_id</code>	Métadonnées seulement — les fichiers vivent en stockage objet (B9)
<code>notification_rules</code>	<code>trigger_point</code> , <code>offset_days</code> , <code>channel</code>	Implémente le moteur J-7/J-3/J/J+1/J+7 (section 41)
<code>notifications</code>	<code>channel</code> , <code>status</code> , <code>payload</code>	File d'envoi, alimentée par Celery (B7)
<code>activity_logs</code>	<code>action</code> , <code>entity_type</code> , <code>metadata</code>	Append-only, jamais modifiée ni supprimée

E6. Domaine 5 — Modèle SaaS (Partie D)

organizations

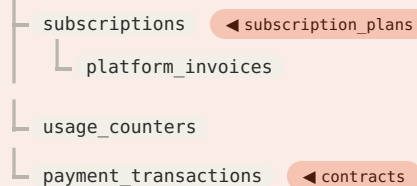


Table	Champs clés	Rôle
<code>subscription_plans</code>	<code>code</code> , <code>price_amount</code> , <code>max_units</code> , <code>features</code>	Les 4 paliers harmonisés en D1
<code>subscriptions</code>	<code>plan_id</code> , <code>status</code> , <code>current_period_end</code>	Un abonnement actif par organisation
<code>platform_invoices</code>	<code>subscription_id</code> , <code>amount</code> , <code>status</code>	Facturation SaaS — distincte de <code>invoices</code> (voir E0)
<code>usage_counters</code>	<code>period</code> , <code>metric</code> , <code>count</code>	Suit la consommation du quota de notifications/SMS

Table	Champs clés	Rôle
<code>payment_transactions</code>	<code>provider</code> , <code>idempotency_key</code> , <code>direction</code>	Trace chaque appel Orange Money/MTN MoMo, mode facilitateur (D3)
Administration plateforme	<code>platform_admins</code>	Comptes internes à l'éditeur, hors périmètre multi-tenant (A15)
	<code>platform_audit_logs</code>	Journal des actions super-admin, distinct de <code>activity_logs</code>
	<code>impersonation_sessions</code>	Trace chaque session « en tant que » un client
	<code>support_tickets</code>	File de support (précise la section 27 du document v1.0)

E7. Isolation multi-tenant : Row Level Security, testée

Le mécanisme, tel que décidé en B3 : chaque requête Django ouvre sa transaction avec `SET LOCAL app.current_org_id = '<uuid>'` (le middleware le fait à partir du JWT décodé), et chaque table sensible porte une politique de ce type :

```
CREATE OR REPLACE FUNCTION current_org_id() RETURNS uuid AS $$
-- NULLIF gère le cas où le paramètre n'a jamais été fixé dans la session :
-- Postgres renvoie alors une chaîne vide plutôt que NULL pour un GUC personnalisé
-- inconnu. Sans ce NULLIF, un middleware distraait ferait planter la requête
-- au lieu de la bloquer silencieusement – on préfère "fail closed" à une erreur 500.
SELECT NULLIF(current_setting('app.current_org_id', true), '')::uuid;
$$ LANGUAGE sql STABLE;

ALTER TABLE properties ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON properties
USING (organization_id = current_org_id());
```



Ce mécanisme a été vérifié sur une instance PostgreSQL 16 réelle, pas seulement relu :

Test	Résultat
Le script <code>schema.sql</code> s'exécute intégralement sans erreur (36 tables, index, triggers, politiques)	Réussi
Un trigger <code>sync_unit_organization_id</code> déduit correctement <code>organization_id</code> d'un <code>unit</code> inséré sans cette colonne	Réussi — valeur héritée automatiquement de la <code>property</code>
Organisation A, avec <code>app.current_org_id</code> fixé sur son UUID, ne voit que sa propre propriété	Réussi
Organisation B, avec son propre UUID, ne voit que la sienne	Réussi
Aucun <code>app.current_org_id</code> fixé dans la transaction	Bug trouvé puis corrigé — la requête plantait au lieu de renvoyer zéro ligne ; corrigé par le <code>NULLIF</code> ci-dessus
La contrainte <code>CHECK</code> interdit de désactiver le 2FA d'un <code>platform_admin</code>	Réussi — tentative de désactivation rejetée par PostgreSQL
<code>support_tickets</code> reste isolé par organisation malgré son lien avec <code>platform_admins</code>	Réussi — une organisation B ne voit aucun ticket d'une organisation A

E8. Stratégie d'index

Au-delà des clés étrangères (indexées systématiquement), trois index méritent d'être soulignés :

- `idx_invoices_due_date` : index partiel (`WHERE status IN ('pending','overdue')`) sur (`organization_id`, `due_date`) — sert directement la détection des impayés (section 16), sans balayer les factures déjà soldées.
- `idx_units_status` et `idx_contracts_status` : index composites (`organization_id`, `status`) — servent le tableau de bord (section 8), qui filtre en permanence par statut au sein d'une organisation.
- `idx_activity_logs_organization_id` : composite (`organization_id`, `created_at DESC`) — le journal d'activité est presque toujours consulté par organisation, trié du plus récent au plus ancien.

E9. Fichier de référence exécutable

Le détail exhaustif — chaque colonne, chaque type, chaque contrainte `CHECK`, chaque index — vit dans `schema.sql`, livré séparément et exécuté avec succès sur PostgreSQL 16 (voir E7). Ce document reste la référence conceptuelle ; `schema.sql` reste la référence exécutable. En cas de divergence future entre les deux, `schema.sql` fait foi.

E10. Domaine 6 — Administration de la plateforme (A15, B18)

Ces tables vivent **hors du périmètre multi-tenant** : elles ne portent volontairement aucun `organization_id` de premier niveau et ne sont jamais soumises aux politiques RLS des tables métier — seul un rôle PostgreSQL séparé avec `BYPASSRLS` (B18) y accède.



Table	Champs clés	Rôle
<code>platform_admins</code>	<code>role</code> (support/billing_admin/super_admin), <code>two_factor_enabled</code>	Comptes internes à l'éditeur, 2FA obligatoire (B18)
<code>platform_audit_logs</code>	<code>admin_id</code> , <code>action</code> , <code>target_organization_id</code>	Journal dédié aux actions super-admin, distinct de <code>activity_logs</code>
<code>impersonation_sessions</code>	<code>admin_id</code> , <code>organization_id</code> , <code>reason</code> , <code>started_at</code> / <code>ended_at</code>	Trace chaque session « en tant que » un client
<code>support_tickets</code>	<code>organization_id</code> , <code>status</code> , <code>priority</code> , <code>assigned_admin_id</code>	File de support (précise la section 27 du document v1.0)



Conclusion

Le document v1.0 pose un socle fonctionnel solide, mais cinq angles morts limitaient sa capacité à devenir un vrai produit exploitable : l'absence d'un modèle économique agence-propriétaire complet (commission, portail, versement), l'absence de toute décision d'architecture concrète sur la partie qui rendra — ou non — le produit tenable techniquement (multi-tenance, asynchrone, paiements mobiles, hors-ligne), l'absence d'un cadrage réglementaire pour le levier de revenu le plus prometteur (la commission sur les paiements), l'absence d'un schéma de données concret sur lequel bâtir tout le reste, et l'absence de toute réflexion sur l'administration de la plateforme elle-même — un besoin qui existe dès le premier client payant, pas seulement à grande échelle.



Ce dernier point n'est plus une intention : le schéma de la Partie E a été exécuté avec succès sur une instance PostgreSQL 16 réelle, isolation multi-tenant comprise. La priorité absolue avant tout code applicatif ou toute communication commerciale : partir de `schema.sql` tel quel, cadrer le module portail propriétaire + commission d'agence (A3, A4), et sécuriser juridiquement le mode de monétisation des paiements (D3) avant de le promettre à un client.